

Who Wrote It? Identifying LLMs from Their Responses

Team

Team leader: Aarush Durgavarjhula

Members:

1. Introduction

The goal of this project is to test whether machine learning models can identify which large language model generated a text response. The classification task uses responses from three LLM families: GPT, Claude, and Gemini. Each model answered the same set of prompts, and the trained classifiers attempt to predict the correct LLM label from the text.

This is a supervised multiclass classification problem. Each example contains a prompt, a generated response, and a label showing which LLM produced the response. The project focuses on two model architectures covered in class: a Convolutional Neural Network (CNN) and a recurrent model implemented with an LSTM.

2. Research Questions

RQ1: Can we identify which LLM generated a response using only the LLM output?

To answer this question, the CNN and LSTM were trained using only the LLM_output field. This tests whether the generated responses contain enough stylistic or structural information to distinguish GPT, Claude, and Gemini.

RQ2: Does the user's prompt help identify the LLM?

To answer this question, each model was trained in three modes:

Mode	Text Used
Input only	LLM_Input
Output only	LLM_output
Input + Output	LLM_Input combined with LLM_output

The input-only experiment tests whether the prompt text alone contains information about the model label. Since all three LLMs answered the same prompts, this mode is not expected to perform well. The output-only and input+output modes test whether the generated text is more useful for classification.

3. Dataset Curation

The final dataset contains 270 total examples. It was created from 90 prompts answered by three LLM families:

LLM family	Number of responses
GPT	90
Claude	90
Gemini	90
Total	270

The prompts were split across six categories:

Category	Prompts per model
general_knowledge	15
science	15
math_reasoning	15
coding	15
creative_writing	15
advice_explanation	15

Each row in the dataset uses the following columns:

Column	Description
prompt_id	Numeric ID for the prompt
category	Prompt category
LLM_name	Classification label: GPT, Claude, or Gemini
LLM_Input	Prompt given to the model
LLM_output	Response generated by the model

The dataset is balanced because each LLM has the same number of responses. The prompts are also aligned across the three models, meaning prompt ID 1 refers to the same prompt for GPT, Claude, and Gemini. Before training, the dataset was checked for missing prompt IDs, blank outputs, duplicate model/prompt pairs, and inconsistent labels.

4. Methodology

4.1 Preprocessing

The text was lowercased and tokenized using a simple regular-expression tokenizer. A vocabulary was built from the training split, and each token was converted into a numeric ID. Sequences were limited to a maximum length of 300 tokens. During batching, shorter sequences were padded so that examples in the same batch had the same length.

The dataset was split into training, validation, and test sets. The split was stratified by the LLM label so that GPT, Claude, and Gemini remained balanced across the training, validation, and test portions.

4.2 CNN Model

The CNN model uses an embedding layer followed by a one-dimensional convolution over the token sequence. The convolution is followed by a ReLU activation, adaptive max pooling, and a final fully connected classification layer. This architecture is useful for text classification because it can detect local word and phrase patterns that may appear more often in one model's responses than another's.

4.3 LSTM Model

The LSTM model uses an embedding layer followed by an LSTM recurrent layer. The final hidden state is passed into a fully connected layer to predict the LLM label. This architecture is useful for text because it processes the sequence in order and can capture longer-range dependencies than a simple bag-of-words model.

4.4 Training

Both models were trained with cross-entropy loss because the task is multiclass classification. The optimizer was Adam with a learning rate of 0.001. Each experiment was trained for 10 epochs with a batch size of 16. Six total experiments were run: CNN and LSTM each using input-only, output-only, and input+output text.

5. Results

5.1 Accuracy Summary

Text mode	CNN test accuracy	LSTM test accuracy
Input only	0.00%	7.32%
Output only	46.34%	36.59%
Input + Output	41.46%	31.71%

The best-performing experiment was the CNN using output-only text, with a test accuracy of 46.34%. The LSTM also performed best in the output-only setting, but its accuracy was lower at 36.59%.

5.2 RQ1 Results: Output-Only Classification

For RQ1, the main comparison is between CNN output-only and LSTM output-only.

Model	Test loss	Test accuracy
CNN output-only	1.0740	46.34%
LSTM output-only	1.1554	36.59%

The CNN performed better than the LSTM for output-only classification. This suggests that local patterns in the generated responses, such as wording choices, sentence structure, or formatting habits, may be useful for identifying the source LLM. However, the accuracy is still not very high, which shows that GPT, Claude, and Gemini responses can be difficult to distinguish with a small dataset.

CNN output-only confusion matrix:

Actual \ Predicted	Claude	GPT	Gemini
Claude	7	4	2
GPT	5	6	3
Gemini	4	4	6

LSTM output-only confusion matrix:

Actual \ Predicted	Claude	GPT	Gemini
Claude	2	10	1
GPT	2	12	0
Gemini	1	12	1

The LSTM output-only model predicted GPT very often. This caused it to correctly identify many GPT examples, but it struggled with Claude and Gemini.

5.3 RQ2 Results: Input vs Output vs Input+Output

The input-only experiments performed very poorly. The CNN input-only model had 0.00% test accuracy, and the LSTM input-only model had 7.32% test accuracy. This makes sense because the same prompts were used for all three LLMs. Since the prompt text is shared across GPT, Claude, and Gemini, it does not provide a meaningful signal for identifying which model wrote the response.

The output-only setting performed best for both architectures. This supports the idea that the generated response contains more useful identifying information than the prompt. Adding the prompt to the output did not improve performance in this experiment. For CNN, accuracy dropped from 46.34% to 41.46%. For LSTM, accuracy dropped from 36.59% to 31.71%.

This result suggests that adding the input may have introduced extra noise. Because the prompt is the same across all three model classes, including it may distract the classifier from the more useful response-specific patterns.

5.4 Loss Curves and Training Behavior

The results folder contains loss graphs for all six experiments:

- cnn_output_loss.png
- cnn_input_loss.png
- cnn_both_loss.png
- lstm_output_loss.png
- lstm_input_loss.png
- lstm_both_loss.png

The output-only and input+output experiments produced more meaningful validation behavior than the input-only experiments. The input-only results show that the models could not learn a useful relationship between the prompts and the LLM labels. This is expected because each prompt appears once for each LLM.

6. Discussion

The results show that identifying LLM-generated text is possible, but difficult with a small dataset. The CNN output-only model was the strongest overall model, reaching 46.34% accuracy on a three-class classification task. This is above random chance, which would be around 33.33%, but not high enough to be considered highly reliable.

The CNN likely performed better because it can detect local textual features. Examples may include common phrase patterns, formatting habits, transitions, or sentence-level style. The LSTM did not perform as well and tended to overpredict GPT in some experiments, especially in the output-only and input+output settings.

The RQ2 experiments showed that the prompt alone is not useful for identifying the model. This is an important result because it confirms that the classifier should focus on model-generated text rather than shared input prompts. The input+output setting also did not improve accuracy, likely because the prompt text adds repeated information that is not tied to the class label.

One limitation is the dataset size. Although the dataset is balanced, 270 total examples is small for training neural networks. More prompts and more responses per model would likely improve the reliability of the results. Another limitation is that only one response per model was collected for each prompt. If multiple

responses were collected per prompt, the model could learn more stable patterns for each LLM family.

7. Optional Extra-Credit Analysis

A useful extension would be to compare response characteristics across GPT, Claude, and Gemini. Possible features include average response length, vocabulary diversity, number of bullet points, use of headings, and formatting style. These features could help explain why the CNN was able to classify some responses better than others.

Another possible extension would be to train a simpler baseline model, such as logistic regression or naive Bayes, using TF-IDF features. This would make it easier to compare whether the neural models are actually providing an advantage over traditional text classification methods.

8. Lessons and Experience

This project showed that dataset structure is very important for machine learning. The input-only experiments performed poorly because the prompts were shared across classes, so the model could not use them to identify the LLM. The output-only results were more useful because the responses contained the actual model-generated language.

The project also showed the importance of comparing multiple architectures and input settings. The CNN outperformed the LSTM in this experiment, especially in the output-only setting. However, both models were limited by the small dataset size. The results demonstrate that LLM fingerprinting is possible, but more data and deeper feature analysis would be needed for a stronger classifier.

9. Conclusion

The project successfully trained and evaluated CNN and LSTM models to classify responses from GPT, Claude, and Gemini. The best result came from the CNN using only the LLM output, with a test accuracy of 46.34%. The LSTM output-only model reached 36.59% accuracy. Input-only classification performed poorly, confirming that the prompt itself does not help identify the LLM when all models answer the same prompts.

Overall, the experiments answer both research questions. For RQ1, the generated output can provide some signal for identifying the LLM, but the classifier is not yet highly accurate. For RQ2, output-only text was more useful than input-only or input+output text. The final results suggest that LLM writing style can be partially detected, but a larger dataset and additional analysis would be needed to make the classifier more reliable.